Reviewer Pack — Minimal Tropical Symplectic Volume and DPLL Proof Tree Width…

Entry 668eeaa777ea · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-05 00:30:47 UTC

Minimal Tropical Symplectic Volume and DPLL Proof Tree Width Inequality

Entry ID: `668eeaa777ea` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-05 00:30:47 UTC

1. Conjecture

Field A (mathematical branch): Tropical Geometry **Field B** (complexity object): Complexity Theory (DPLL Search Trees)

Statement:

> For every satisfiability instance φ with n variables, the minimal tropical symplectic volume (TSV(φ)) of its associated matroid is Ω(w_DPLL(φ)), where w_DPLL(φ) is the width of the DPLL proof tree for φ.

Rationale (proposer's reasoning):

> Tropical geometry provides a framework to study the structure of computational problems through geometric objects. The symplectic volume, as an invariant in tropical geometry, could potentially capture the complexity of the search space in the DPLL algorithm, providing insights into its performance.

Taxonomy category: `TROPICAL_GEOMETRY` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `b870273468d31262`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if, for all generated satisfiability instances φ with n variables (n ≤ 40), the ratio of the tropical symplectic volume TSV(M(φ)) to the DPLL proof tree width w_DPLL(φ) is greater than or equal to 1.0 across at least 25 out of 30 seeds.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	1.00	UNCERTAIN	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.90	UNCERTAIN	SAFE
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "tropical geometry" AND "DPLL proof tree width" - "minimal tropical symplectic volume" AND "satisfiability instance" - "matroid" AND "width of DPLL proof tree"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def gaussian_elimination(A):
    n = len(A)
    for i in range(n):
        max_row = i
        for j in range(i+1, n):
            if abs(A[j][i]) > abs(A[max_row][i]):
                max_row = j
        A[i], A[max_row] = A[max_row], A[i]
        if A[i][i] == 0:
            continue
        for j in range(n):
            if j != i:
                factor = Fraction(A[j][i], A[i][i])
                for k in range(n+1):
                    A[j][k] -= factor * A[i][k]
    return A

def tsv(matroid):
    n = len(matroid)
    A = [[0]*n for _ in range(n)]
    for i in range(n):
        for j in range(i, n):
            if matroid[i][j]:
                A[i][j] = 1
                A[j][i] = 1
    rank = sum(1 for row in gaussian_elimination(A) if any(row))
    return math.factorial(rank)

def dpll_width(clauses, assignment):
    stack = [(clauses, assignment)]
```

```
    while stack:
        clauses, assignment = stack.pop()
        if not clauses:
            return 0
        unit_clause = next((c for c in clauses if len(c) == 1), None)
        if unit_clause:
            literal = unit_clause[0]
            new_assignment = assignment.copy()
            new_assignment[literal] = True
            new_clauses = [c for c in clauses if literal not in c and -literal not in c]
            stack.append((new_clauses, new_assignment))
            continue
        pure_literal = next((l for l in range(1, len(assignment)+1) if (l not in assignment and -l
        if pure_literal:
            new_assignment = assignment.copy()
            new_assignment[pure_literal] = True
            new_clauses = [c for c in clauses if pure_literal not in c and -pure_literal not in c]
            stack.append((new_clauses, new_assignment))
            continue
        literal = random.choice([l for l in range(1, len(assignment)+1) if l not in assignment])
        new_assignment_true = assignment.copy()
        new_assignment_true[literal] = True
        new_clauses_true = [c for c in clauses if literal not in c and -literal not in c]
        stack.append((new_clauses_true, new_assignment_true))
        new_assignment_false = assignment.copy()
        new_assignment_false[literal] = False
        new_clauses_false = [c for c in clauses if literal not in c and -literal not in c]
        stack.append((new_clauses_false, new_assignment_false))
    return 1

def generate_instance(n):
    clauses = []
    for i in range(1, n+1):
        clause = random.sample(range(1, n+1), random.randint(1, n))
        clauses.append(clause)
    return clauses

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = 40
    instances_tested = 30
    n_max = 40
    conjecture_holds = True
    counterexample = ""

    for _ in range(instances_tested):
        clauses = generate_instance(n)
        matroid = [[any(l in c or -l in c for c in clauses) for l in range(1, n+1)] for _ in range
        tsv_value = tsv(matroid)
        dpll_value = dpll_width(clauses, {})

        if dpll_value == 0:
            continue

        ratio = Fraction(tsv_value, dpll_value)
        if ratio < 1.0:
```

```
            conjecture_holds = False
            counterexample = f"Instance with TSV={tsv_value} and DPLL width={dpll_value}"

    return {
        "metric_name": "Ratio of TSV to DPLL Width",
        "metric_value": Fraction(tsv_value, dpll_value) if dpll_value != 0 else None,
        "instances_tested": instances_tested,
        "n_max": n_max,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results if r["metric_value"] is not None) / len(re
    std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results if r["metric_va
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    else:
        first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
        print(f"RESULT: FALSIFIED counterexample=\"{results[first_failing_seed]['counterexample']}
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_21ecbe43.py", line 122, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_21ecbe43.py", line 96, in run_trial
    tsv_value = tsv(matroid)
                ^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_21ecbe43.py", line 43, in tsv
    rank = sum(1 for row in gaussian_elimination(A) if any(row))
                            ^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_21ecbe43.py", line 32, in gaussian_elim
    A[j][k] -= factor * A[i][k]
    ~~~~^^^
IndexError: list index out of range
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The test code crashed before producing data, which prevents us from verifying the conjecture's support condition. | next: Debug the crashing issue in the test code to ensure it can complete and verify the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	21191
2	preregistration	ollama_remote	glm4:latest	0	0	9608
3	novelty	ollama_remote	glm4:latest	0	0	8307
4	novelty	ollama_remote	glm4:latest	0	0	9190
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	45990
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14683
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	38535
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17791
9	judge	ollama_remote	glm4:latest	0	0	62092

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 227387 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/668eeaa777ea.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/668eeaa777ea.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/668eeaa777ea.tar.gz` (if generated)